Problem 1: Bubble Sort

Approach. Compare neighbouring elements and swap them if they are out of order. Each pass moves the largest unsorted value to the end. If a pass makes no swap, the array is sorted and the loop stops.

Algorithm 1 Bubble Sort

Input: An array A of n numbers
Output: A sorted in increasing order

for $last \leftarrow n-1$ **downto** 1 **do**
    $swapped \leftarrow$ **false**
    for $j \leftarrow 0$ **to** $last-1$ **do**
        if $A[j] > A[j+1]$ **then**
            $\textsc{Swap}(A[j], A[j+1])$
            $swapped \leftarrow$ **true**
        end if
    end for
    $\textsc{PrintArray}(A, n)$
    if $swapped =$ **false then**
        break
    end if
end for

C++ code.

```cpp
#include <iostream>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

void bubbleSort(int arr[], int n, int &passes, int &comparisons, int &swaps) {
    passes = 0;
    comparisons = 0;
    swaps = 0;

    for (int last = n - 1; last > 0; last--) {
        bool swapped = false;
        for (int j = 0; j < last; j++) {
            comparisons++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swaps++;
                swapped = true;
            }
        }
        passes++;
        cout << "Pass " << passes << ":   ";
        printArray(arr, n);

        if (!swapped) break;
    }
```

```
}

int main() {
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int passes, comparisons, swaps;

    cout << "=== Run 1: original array A ===" << endl;
    cout << "Original: ";
    printArray(A, n);
    bubbleSort(A, n, passes, comparisons, swaps);
    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Passes: " << passes << "   Comparisons: " << comparisons
         << "   Swaps: " << swaps << endl;

    int sortedCopy[8];                  // A is sorted now
    for (int k = 0; k < n; k++) sortedCopy[k] = A[k];

    cout << endl << "=== Run 2: already sorted copy of A ===" << endl;
    cout << "Original: ";
    printArray(sortedCopy, n);
    bubbleSort(sortedCopy, n, passes, comparisons, swaps);
    cout << "Sorted:   ";
    printArray(sortedCopy, n);
    cout << "Passes: " << passes << "   Comparisons: " << comparisons
         << "   Swaps: " << swaps << endl;

    return 0;
}
```

Output.

```
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
Pass 1:   7 23 32 5 34 14 19 62
Pass 2:   7 23 5 32 14 19 34 62
Pass 3:   7 5 23 14 19 32 34 62
Pass 4:   5 7 14 19 23 32 34 62
Pass 5:   5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Passes: 5   Comparisons: 25   Swaps: 15

=== Run 2: already sorted copy of A ===
Original: 5 7 14 19 23 32 34 62
Pass 1:   5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Passes: 1   Comparisons: 7   Swaps: 0
```

Item 5. On the sorted copy: 1 pass, 7 comparisons, 0 swaps. No pair is out of order, so the first pass makes no swap, *swapped* stays false, and the loop stops.

Problem 2: Insertion Sort

Approach. Take each element as the key, shift the larger elements of the sorted part one position right, and insert the key into the gap.

Algorithm 2 Insertion Sort

Input: An array A of n numbers
Output: A sorted in increasing order

for $i \leftarrow 1$ **to** $n-1$ **do**
    $key \leftarrow A[i]$
    $j \leftarrow i-1$
    while $j \geq 0$ **and** $A[j] > key$ **do**
        $A[j+1] \leftarrow A[j]$
        $j \leftarrow j-1$
    end while
    $A[j+1] \leftarrow key$
    Print i, key and PrintArray(A, n)
end for

C++ code.

```cpp
#include <iostream>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

void insertionSort(int arr[], int n, int &comparisons, int &shifts) {
    comparisons = 0;
    shifts = 0;

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;      // signed: j reaches -1 when key is a new minimum

        while (j >= 0) {
            comparisons++;
            if (arr[j] <= key) break;
            arr[j + 1] = arr[j];
            shifts++;
            j--;
        }
        arr[j + 1] = key;

        cout << "i = " << i << ", key = " << key << ": ";
        printArray(arr, n);
    }
}

int main() {
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisons, shifts;

    cout << "=== Run 1: original array A ===" << endl;
```

```
    cout << "Original: ";
    printArray(A, n);
    insertionSort(A, n, comparisons, shifts);
    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Comparisons: " << comparisons << "   Shifts: " << shifts << endl;

    // A is sorted now
    int B[8], C[8];
    for (int k = 0; k < n; k++) {
        B[k] = A[k];
        C[k] = A[n - 1 - k];
    }

    cout << endl << "=== Run 2: ascending order (fewest shifts) ===" << endl;
    cout << "Original: ";
    printArray(B, n);
    insertionSort(B, n, comparisons, shifts);
    cout << "Sorted:   ";
    printArray(B, n);
    cout << "Comparisons: " << comparisons << "   Shifts: " << shifts << endl;

    cout << endl << "=== Run 3: descending order (most shifts) ===" << endl;
    cout << "Original: ";
    printArray(C, n);
    insertionSort(C, n, comparisons, shifts);
    cout << "Sorted:   ";
    printArray(C, n);
    cout << "Comparisons: " << comparisons << "   Shifts: " << shifts << endl;

    return 0;
}
```

Output.

```
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 20   Shifts: 15

=== Run 2: ascending order (fewest shifts) ===
Original: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 7   Shifts: 0

=== Run 3: descending order (most shifts) ===
Original: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 28   Shifts: 28
```

Item 4. Fewest shifts: ascending order 5 7 14 19 23 32 34 62 (0 shifts). Most shifts: descending order 62 34 32 23 19 14 7 5 (28 shifts). Each shift fixes one pair that is out of order; ascending order has no such pair and descending order has all $n(n-1)/2 = 28$ of them.

Problem 3: Selection Sort

Approach. In each pass, find the smallest value in the unsorted part and swap it into the first unsorted position. Skip the swap if it is already there.

Algorithm 3 Selection Sort

Input: An array A of n numbers
Output: A sorted in increasing order

for $i \leftarrow 0$ **to** $n-2$ **do**
    $minIdx \leftarrow i$
    for $j \leftarrow i+1$ **to** $n-1$ **do**
        if $A[j] < A[minIdx]$ **then**
            $minIdx \leftarrow j$
        end if
    end for
    if $minIdx \neq i$ **then**
        Swap$(A[i], A[minIdx])$
    end if
    Print $A[i]$, $minIdx$ and PrintArray(A, n)
end for

C++ code.

```cpp
#include <iostream>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

void selectionSort(int arr[], int n, int &comparisons, int &swaps) {
    comparisons = 0;
    swaps = 0;

    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            comparisons++;
            if (arr[j] < arr[minIdx]) minIdx = j;
        }

        int minVal = arr[minIdx];
        if (minIdx != i) {
            int temp = arr[i];
            arr[i] = arr[minIdx];
            arr[minIdx] = temp;
            swaps++;
        }

        cout << "Pass " << i + 1 << ": min = " << minVal
             << " at index " << minIdx << " -> ";
        printArray(arr, n);
    }
}

```

```
int main() {
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisons, swaps;

    cout << "Original: ";
    printArray(A, n);
    selectionSort(A, n, comparisons, swaps);
    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Comparisons: " << comparisons << "   Swaps: " << swaps << endl;

    return 0;
}
```

Output.

```
Original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 28   Swaps: 5
```

Item 4. Selection Sort makes 5 swaps on A and Bubble Sort makes 15. Selection Sort makes at most one swap per pass, because it finds the minimum first and moves it directly to its final position. Bubble Sort only swaps neighbours, and each swap fixes only one pair that is out of order, so it needs one swap for each of the 15 such pairs in A.

Problem 4: Quick Sort

Approach. Use the last element as the pivot. The partition moves the smaller values to the left of the pivot and puts the pivot in its final position. Then sort the left and right parts recursively.

Algorithm 4 Quick Sort (Lomuto partition)

Input: An array A and indices low, $high$
Output: $A[low..high]$ sorted in increasing order

procedure QUICKSORT($A, low, high, depth$)
  if $low < high$ **then**
    Print $depth$ indentation, $A[high]$ and PRINTARRAY($A[low..high]$)
    $p \leftarrow$ PARTITION($A, low, high$)
    QUICKSORT($A, low, p-1, depth+1$)
    QUICKSORT($A, p+1, high, depth+1$)
  end if
end procedure
procedure PARTITION($A, low, high$)
  $pivot \leftarrow A[high]$
  $i \leftarrow low - 1$
  for $j \leftarrow low$ **to** $high - 1$ **do**
    if $A[j] < pivot$ **then**
      $i \leftarrow i + 1$
      SWAP($A[i], A[j]$)
      Print i, j and PRINTARRAY($A[low..high]$) (first partition only)
    end if
  end for
  SWAP($A[i+1], A[high]$)
  Print the pivot position $i + 1$ (first partition only)
  return $i + 1$
end procedure

C++ code.

```cpp
#include <iostream>
#include <string>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

int partition(int arr[], int low, int high, int &comparisons, bool showSteps) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        comparisons++;
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
```

```
            arr[j] = temp;
            if (showSteps) {
                cout << "  | swap  i = " << i << ", j = " << j << ": ";
                printArray(arr + low, high - low + 1);
            }
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    if (showSteps) {
        cout << "  | pivot " << pivot << " placed at index " << i + 1 << ": ";
        printArray(arr + low, high - low + 1);
    }
    return i + 1;
}

void quickSort(int arr[], int low, int high, int &comparisons, int depth = 0) {
    if (low >= high) return;

    cout << string(3 * depth, ' ') << "pivot = " << arr[high] << ": ";
    printArray(arr + low, high - low + 1);

    int p = partition(arr, low, high, comparisons, depth == 0);
    quickSort(arr, low, p - 1, comparisons, depth + 1);
    quickSort(arr, p + 1, high, comparisons, depth + 1);
}

int main() {
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisons;

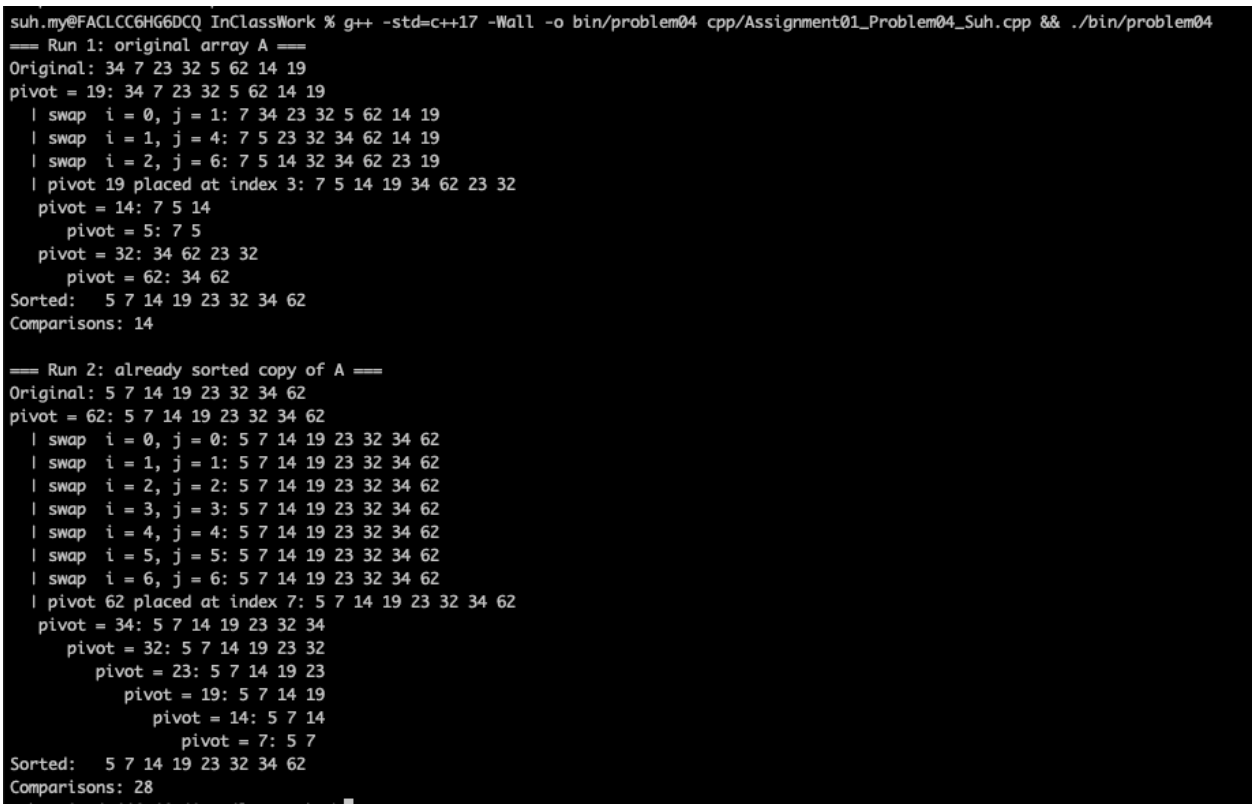
    cout << "=== Run 1: original array A ===" << endl;
    cout << "Original: ";
    printArray(A, n);
    comparisons = 0;
    quickSort(A, 0, n - 1, comparisons);
    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Comparisons: " << comparisons << endl;

    int sortedCopy[8];                  // A is sorted now
    for (int k = 0; k < n; k++) sortedCopy[k] = A[k];

    cout << endl << "=== Run 2: already sorted copy of A ===" << endl;
    cout << "Original: ";
    printArray(sortedCopy, n);
    comparisons = 0;
    quickSort(sortedCopy, 0, n - 1, comparisons);
    cout << "Sorted:   ";
    printArray(sortedCopy, n);
    cout << "Comparisons: " << comparisons << endl;

    return 0;
}
```

Output.



Item 5. On the sorted copy Quick Sort makes 28 comparisons instead of 14. In a sorted array the last element is always the largest, so each partition leaves all the other elements on one side and the problem shrinks by only one element per call: $7 + 6 + \cdots + 1 = n(n-1)/2 = 28$ comparisons, $O(n^2)$. To avoid it, choose the pivot at random or use the median of the first, middle and last elements.

Problem 5: Merge Sort

Approach. Split the array at $mid = (low + high)/2$, sort each half recursively, and merge the two sorted halves by repeatedly taking the smaller front element.

Algorithm 5 Merge Sort

Input: An array A and indices low, $high$
Output: $A[low..high]$ sorted in increasing order

procedure MERGESORT($A, low, high$)
  if $low < high$ **then**
    Print "split:" and PRINTARRAY($A[low..high]$)
    $mid \leftarrow \lfloor (low + high)/2 \rfloor$
    MERGESORT(A, low, mid)
    MERGESORT($A, mid + 1, high$)
    MERGE($A, low, mid, high$)
    Print "merged:" and PRINTARRAY($A[low..high]$)
  end if
end procedure
procedure MERGE($A, low, mid, high$)
  $L \leftarrow A[low..mid]$; $R \leftarrow A[mid + 1..high]$
  $i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow low$
  while $i < |L|$ **and** $j < |R|$ **do**
    if $L[i] \leq R[j]$ **then**
      $A[k] \leftarrow L[i]$; $i \leftarrow i + 1$
    else
      $A[k] \leftarrow R[j]$; $j \leftarrow j + 1$
    end if
    $k \leftarrow k + 1$
  end while
  Copy the rest of L and R into $A[k..high]$
end procedure

C++ code.

```cpp
#include <iostream>
#include <vector>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

void merge(int arr[], int low, int mid, int high, int &comparisons) {
    vector<int> left(arr + low, arr + mid + 1);
    vector<int> right(arr + mid + 1, arr + high + 1);
    int nLeft = left.size(), nRight = right.size();
    int i = 0, j = 0, k = low;

    while (i < nLeft && j < nRight) {
        comparisons++;
```

```
        if (left[i] <= right[j]) arr[k++] = left[i++];   // <= (not <) keeps the sort stable
        else                     arr[k++] = right[j++];
    }
    while (i < nLeft)  arr[k++] = left[i++];
    while (j < nRight) arr[k++] = right[j++];
}

// The top-level merge runs last, so finalMerge ends up holding its count
void mergeSort(int arr[], int low, int high, int &comparisons, int &finalMerge) {
    if (low >= high) return;

    cout << "split:  ";
    printArray(arr + low, high - low + 1);

    int mid = (low + high) / 2;
    mergeSort(arr, low, mid, comparisons, finalMerge);
    mergeSort(arr, mid + 1, high, comparisons, finalMerge);

    int before = comparisons;
    merge(arr, low, mid, high, comparisons);
    finalMerge = comparisons - before;

    cout << "merged: ";
    printArray(arr + low, high - low + 1);
}

int main() {
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int comparisons, finalMerge;

    cout << "=== Run 1: original array A ===" << endl;
    cout << "Original: ";
    printArray(A, n);
    comparisons = 0;
    finalMerge = 0;
    mergeSort(A, 0, n - 1, comparisons, finalMerge);
    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Comparisons: " << comparisons << "   (final merge: " << finalMerge << ")" << endl;

    // A is sorted now
    int sortedCopy[8], reversedCopy[8];
    for (int k = 0; k < n; k++) {
        sortedCopy[k] = A[k];
        reversedCopy[k] = A[n - 1 - k];
    }

    cout << endl << "=== Run 2: sorted copy of A ===" << endl;
    cout << "Original: ";
    printArray(sortedCopy, n);
    comparisons = 0;
    finalMerge = 0;
    mergeSort(sortedCopy, 0, n - 1, comparisons, finalMerge);
    cout << "Sorted:   ";
    printArray(sortedCopy, n);
    cout << "Comparisons: " << comparisons << "   (final merge: " << finalMerge << ")" << endl;

    cout << endl << "=== Run 3: reversed copy of A ===" << endl;
    cout << "Original: ";
    printArray(reversedCopy, n);
    comparisons = 0;
    finalMerge = 0;
    mergeSort(reversedCopy, 0, n - 1, comparisons, finalMerge);
    cout << "Sorted:   ";
    printArray(reversedCopy, n);
    cout << "Comparisons: " << comparisons << "   (final merge: " << finalMerge << ")" << endl;

```

```
89      return 0;
90  }
```

Output.

```
sun.my@FAC1CC0HG6DCQ InClassWork % g++ -std=c++17 -Wall -o bin/problem05 cpp/Assignment01_Problem05_Sun.cpp && ./bin/problem05
=== Run 1: original array A ===
Original: 34 7 23 32 5 62 14 19
split:  34 7 23 32 5 62 14 19
split:  34 7 23 32
split:  34 7
merged: 7 34
split:  23 32
merged: 23 32
merged: 7 23 32 34
split:  5 62 14 19
split:  5 62
merged: 5 62
split:  14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 17   (final merge: 7)

=== Run 2: sorted copy of A ===
Original: 5 7 14 19 23 32 34 62
split:  5 7 14 19 23 32 34 62
split:  5 7 14 19
split:  5 7
merged: 5 7
split:  14 19
merged: 14 19
merged: 5 7 14 19
split:  23 32 34 62
split:  23 32
merged: 23 32
split:  34 62
merged: 34 62
merged: 23 32 34 62
merged: 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 12   (final merge: 4)

=== Run 3: reversed copy of A ===
Original: 62 34 32 23 19 14 7 5
split:  62 34 32 23 19 14 7 5
split:  62 34 32 23
split:  62 34
merged: 34 62
split:  32 23
merged: 23 32
merged: 23 32 34 62
split:  19 14 7 5
split:  19 14
merged: 14 19
split:  7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
Sorted:   5 7 14 19 23 32 34 62
Comparisons: 12   (final merge: 4)
```

Item 5. Comparisons: 17 for A, 12 for the sorted copy, 12 for the reversed copy. The splits depend only on n, so the same merges happen for every order; the order only changes how early each merge runs out of one half. Bubble Sort (7 to 28) and Quick Sort (14 to 28) change much more because their work depends on the order. Merge Sort needs $O(n)$ extra memory for the temporary arrays used in merging.

Problem 6: Heap Sort

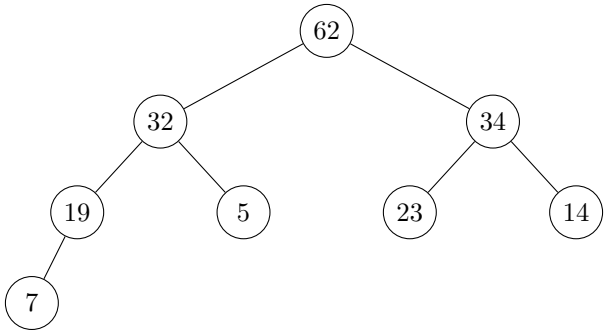
Approach. Build a max-heap, where every parent is at least as large as its children. Then repeatedly swap the root (the maximum) with the last element of the heap, shrink the heap by one, and sift the new root down.

Algorithm 6 Heap Sort

Input: An array A of n numbers
Output: A sorted in increasing order

```
 1: procedure SiftDown(A, root, size)
 2:     loop
 3:         largest ← root;   left ← 2 · root + 1;   right ← 2 · root + 2
 4:         if left < size and A[left] > A[largest] then
 5:             largest ← left
 6:         end if
 7:         if right < size and A[right] > A[largest] then
 8:             largest ← right
 9:         end if
10:         if largest = root then
11:             return
12:         end if
13:         Swap(A[root], A[largest])
14:         root ← largest
15:     end loop
16: end procedure
17: procedure HeapSort(A, n)
18:     for root ← ⌊n/2⌋ − 1 downto 0 do
19:         SiftDown(A, root, n)
20:     end for
21:     Print "Heap:" and PrintArray(A, n)
22:     for end ← n − 1 downto 1 do
23:         Swap(A[0], A[end])
24:         SiftDown(A, 0, end)
25:         PrintArray(A[0..end − 1]);   PrintArray(A[end..n − 1])
26:     end for
27: end procedure
```

Heap after building: 62 32 34 19 5 23 14 7



C++ code.

```cpp
// printArray ends its line, so each pass prints the heap and sorted parts on two lines

#include <iostream>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

void siftDown(int arr[], int root, int size, int &swaps) {
    while (true) {
        int largest = root;
        int left = 2 * root + 1;
        int right = 2 * root + 2;
        if (left < size && arr[left] > arr[largest])   largest = left;
        if (right < size && arr[right] > arr[largest]) largest = right;
        if (largest == root) return;

        int temp = arr[root];
        arr[root] = arr[largest];
        arr[largest] = temp;
        swaps++;
        root = largest;
    }
}

void heapSort(int arr[], int n, int &swaps) {
    swaps = 0;

    // Build the max-heap from the last parent, n/2 - 1, back to the root
    for (int root = n / 2 - 1; root >= 0; root--)
        siftDown(arr, root, n, swaps);
    cout << "Heap:     ";
    printArray(arr, n);

    for (int end = n - 1; end >= 1; end--) {
        int temp = arr[0];                      // not counted: only swaps inside siftDown count
        arr[0] = arr[end];
        arr[end] = temp;
        siftDown(arr, 0, end, swaps);

        cout << "end = " << end << ": heap:   ";
        printArray(arr, end);
        cout << "         sorted: ";
        printArray(arr + end, n - end);
    }
}

int main() {
    int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    int n = sizeof(A) / sizeof(A[0]);
    int swaps;

    cout << "Original: ";
    printArray(A, n);
    heapSort(A, n, swaps);
    cout << "Sorted:   ";
    printArray(A, n);
    cout << "Swaps in siftDown: " << swaps << endl;

    return 0;
}
```

Output.



Item 5. Heap Sort is not stable. Example: $[4_a, 4_b, 1]$, which is already a max-heap. Swapping the root to the end gives $[1, 4_b, 4_a]$, sifting down gives $[4_b, 1, 4_a]$, and the last swap gives $[1, 4_b, 4_a]$. The two equal values end up in reverse order.

Problem 7: Comparing the Six Algorithms

Approach. The program uses the six sorts from Problems 1–6 without printing, and each one counts its comparisons. For timing, every algorithm sorts a fresh copy of the same random data (values 1–100,000, fixed seed), and the time is the average of 5 runs.

Algorithm 7 Comparing the algorithms

```
for each input X ∈ {A, sorted A, reversed A} do
    for each algorithm SORT do
        Sort a copy of X and record the comparisons
    end for
end for
for each n ∈ {1000, 5000, 10000} do
    data ← n random values in [1, 100000]
    for each algorithm SORT do
        Time SORT on a fresh copy of data 5 times and take the average
        PRINTARRAY(work, 10)
    end for
end for
```

Item 1: comparisons.

Algorithm	A	Sorted	Reversed
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	26	27	24

Item 2: running time (ms).

Algorithm	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$	10,000/1,000
Bubble Sort	0.276	5.831	22.107	80.1
Insertion Sort	0.077	1.472	5.685	73.8
Selection Sort	0.789	19.229	73.981	93.7
Quick Sort	0.015	0.124	0.442	29.0
Merge Sort	0.059	0.232	0.750	12.6
Heap Sort	0.023	0.147	0.591	26.0

C++ code.

```cpp
// Compile with -O2 for timing: g++ -std=c++17 -O2 -o compare Assignment01_Problem07_Suh.cpp

#include <iostream>
#include <iomanip>
#include <vector>
#include <random>
#include <chrono>
#include <utility>
```

```cpp
#include <string>
using namespace std;

void printArray(const int arr[], int n) {
    for (int k = 0; k < n; k++) {
        cout << arr[k];
        if (k < n - 1) cout << " ";
    }
    cout << endl;
}

// Problem 1: Bubble Sort
void bubbleSort(int arr[], int n, long long &comparisons) {
    for (int last = n - 1; last > 0; last--) {
        bool swapped = false;
        for (int j = 0; j < last; j++) {
            comparisons++;
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

// Problem 2: Insertion Sort
void insertionSort(int arr[], int n, long long &comparisons) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0) {
            comparisons++;
            if (arr[j] <= key) break;
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Problem 3: Selection Sort
void selectionSort(int arr[], int n, long long &comparisons) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            comparisons++;
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        if (minIdx != i) swap(arr[i], arr[minIdx]);
    }
}

// Problem 4: Quick Sort
int partition(int arr[], int low, int high, long long &comparisons) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        comparisons++;
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

```

```
void quickSort(int arr[], int low, int high, long long &comparisons) {
    if (low >= high) return;
    int p = partition(arr, low, high, comparisons);
    quickSort(arr, low, p - 1, comparisons);
    quickSort(arr, p + 1, high, comparisons);
}

// Problem 5: Merge Sort
void merge(int arr[], int low, int mid, int high, long long &comparisons) {
    vector<int> left(arr + low, arr + mid + 1);
    vector<int> right(arr + mid + 1, arr + high + 1);
    int nLeft = left.size(), nRight = right.size();
    int i = 0, j = 0, k = low;
    while (i < nLeft && j < nRight) {
        comparisons++;
        if (left[i] <= right[j]) arr[k++] = left[i++];
        else                     arr[k++] = right[j++];
    }
    while (i < nLeft)  arr[k++] = left[i++];
    while (j < nRight) arr[k++] = right[j++];
}

void mergeSort(int arr[], int low, int high, long long &comparisons) {
    if (low >= high) return;
    int mid = (low + high) / 2;
    mergeSort(arr, low, mid, comparisons);
    mergeSort(arr, mid + 1, high, comparisons);
    merge(arr, low, mid, high, comparisons);
}

// Problem 6: Heap Sort
void siftDown(int arr[], int root, int size, long long &comparisons) {
    while (true) {
        int largest = root;
        int left = 2 * root + 1;
        int right = 2 * root + 2;
        if (left < size) {
            comparisons++;
            if (arr[left] > arr[largest]) largest = left;
        }
        if (right < size) {
            comparisons++;
            if (arr[right] > arr[largest]) largest = right;
        }
        if (largest == root) return;
        swap(arr[root], arr[largest]);
        root = largest;
    }
}

void heapSort(int arr[], int n, long long &comparisons) {
    for (int root = n / 2 - 1; root >= 0; root--)
        siftDown(arr, root, n, comparisons);
    for (int end = n - 1; end >= 1; end--) {
        swap(arr[0], arr[end]);
        siftDown(arr, 0, end, comparisons);
    }
}

// Wrappers so all six sorts share one signature
void quickSortAll(int arr[], int n, long long &comparisons) { quickSort(arr, 0, n - 1, comparisons); }
void mergeSortAll(int arr[], int n, long long &comparisons) { mergeSort(arr, 0, n - 1, comparisons); }

typedef void (*SortFunction)(int arr[], int n, long long &comparisons);
struct Algorithm {
    const char *name;
    SortFunction sort;
};
```

```
const Algorithm ALGORITHMS[] = {
    {"Bubble Sort", bubbleSort},      {"Insertion Sort", insertionSort},
    {"Selection Sort", selectionSort}, {"Quick Sort", quickSortAll},
    {"Merge Sort", mergeSortAll},      {"Heap Sort", heapSort}};
const int NUM_ALGORITHMS = 6;

bool isSorted(const int arr[], int n) {
    for (int k = 1; k < n; k++)
        if (arr[k - 1] > arr[k]) return false;
    return true;
}

// Sorts a fresh copy of data; returns the time in ms
double timedSort(const Algorithm &algorithm, const vector<int> &data,
                 vector<int> &work, long long &comparisons) {
    work = data;
    comparisons = 0;
    auto start = chrono::steady_clock::now();
    algorithm.sort(work.data(), (int)work.size(), comparisons);
    auto stop = chrono::steady_clock::now();
    return chrono::duration<double, milli>(stop - start).count();
}

int main() {
    // Part 1: comparisons on A, sorted A and reversed A
    const int A[] = {34, 7, 23, 32, 5, 62, 14, 19};
    const int n = sizeof(A) / sizeof(A[0]);
    vector<int> original(A, A + n), sortedA(A, A + n), reversedA(n), work;
    long long unused = 0;
    insertionSort(sortedA.data(), n, unused);      // no library sort allowed
    for (int k = 0; k < n; k++) reversedA[k] = sortedA[n - 1 - k];

    cout << "=== Part 1: comparisons on the eight-element inputs ===" << endl;
    cout << left << setw(16) << "Algorithm" << right << setw(8) << "A"
         << setw(8) << "Sorted" << setw(10) << "Reversed" << endl;
    for (int a = 0; a < NUM_ALGORITHMS; a++) {
        long long cA, cS, cR;
        timedSort(ALGORITHMS[a], original, work, cA);
        timedSort(ALGORITHMS[a], sortedA, work, cS);
        timedSort(ALGORITHMS[a], reversedA, work, cR);
        cout << left << setw(16) << ALGORITHMS[a].name << right << setw(8) << cA
             << setw(8) << cS << setw(10) << cR << endl;
    }

    // Part 2: timing and comparisons on random arrays
    const int SIZES[] = {1000, 5000, 10000};
    const int NUM_SIZES = 3;
    const int REPEATS = 5;
    // mt19937 is fully specified, so every compiler generates the same data
    mt19937 rng(7205);

    double avgMs[NUM_ALGORITHMS][NUM_SIZES];
    long long comps[NUM_ALGORITHMS][NUM_SIZES];
    int firstTen[NUM_ALGORITHMS][NUM_SIZES][10];
    bool allSorted = true;

    // Untimed warm-up; its own generator leaves the measured data unchanged
    mt19937 warmRng(1);
    vector<int> warm(5000);
    for (int &x : warm) x = 1 + warmRng() % 100000;
    for (int a = 0; a < NUM_ALGORITHMS; a++) {
        long long ignored;
        timedSort(ALGORITHMS[a], warm, work, ignored);
    }

    for (int s = 0; s < NUM_SIZES; s++) {
        vector<int> data(SIZES[s]);
        for (int &x : data) x = 1 + rng() % 100000;
```

```
        for (int a = 0; a < NUM_ALGORITHMS; a++) {
            double total = 0;
            for (int r = 0; r < REPEATS; r++)
                total += timedSort(ALGORITHMS[a], data, work, comps[a][s]);
            avgMs[a][s] = total / REPEATS;
            if (!isSorted(work.data(), SIZES[s])) allSorted = false;
            for (int k = 0; k < 10; k++) firstTen[a][s][k] = work[k];
        }
    }

    cout << endl << "=== Part 2a: average running time in ms on random arrays ("
         << REPEATS << " runs each) ===" << endl;
    cout << left << setw(16) << "Algorithm" << right;
    for (int s = 0; s < NUM_SIZES; s++) cout << setw(12) << ("n = " + to_string(SIZES[s]));
    cout << setw(12) << "10000/1000" << setw(12) << "10000/5000" << endl;
    cout << fixed << setprecision(3);
    for (int a = 0; a < NUM_ALGORITHMS; a++) {
        cout << left << setw(16) << ALGORITHMS[a].name << right;
        for (int s = 0; s < NUM_SIZES; s++) cout << setw(12) << avgMs[a][s];
        cout << setw(12) << setprecision(1) << avgMs[a][2] / avgMs[a][0]
             << setw(12) << setprecision(2) << avgMs[a][2] / avgMs[a][1]
             << setprecision(3) << endl;
    }

    cout << endl << "=== Part 2b: comparisons on the same random arrays ===" << endl;
    cout << left << setw(16) << "Algorithm" << right;
    for (int s = 0; s < NUM_SIZES; s++) cout << setw(12) << ("n = " + to_string(SIZES[s]));
    cout << setw(12) << "10000/1000" << setw(12) << "10000/5000" << endl;
    for (int a = 0; a < NUM_ALGORITHMS; a++) {
        cout << left << setw(16) << ALGORITHMS[a].name << right;
        for (int s = 0; s < NUM_SIZES; s++) cout << setw(12) << comps[a][s];
        cout << setw(12) << setprecision(1) << (double)comps[a][2] / comps[a][0]
             << setw(12) << setprecision(2) << (double)comps[a][2] / comps[a][1]
             << setprecision(3) << endl;
    }
    cout << "Theory, 1000 -> 10000: n^2 grows 100x, n log n about 13.3x" << endl
         << "Theory, 5000 -> 10000: n^2 grows 4x,   n log n about 2.16x" << endl;

    // Part 3: first 10 elements of every sorted array
    cout << endl << "=== Part 3: first 10 elements of each sorted array ===" << endl;
    for (int s = 0; s < NUM_SIZES; s++) {
        cout << "n = " << SIZES[s] << endl;
        for (int a = 0; a < NUM_ALGORITHMS; a++) {
            cout << "  " << left << setw(16) << ALGORITHMS[a].name << right;
            printArray(firstTen[a][s], 10);
        }
    }
    cout << "Full check of all " << NUM_ALGORITHMS * NUM_SIZES << " sorted arrays: "
         << (allSorted ? "every element is in order" : "ERROR, an array is out of order")
         << endl;

    // Part 4: almost sorted input
    const int NEAR_N = 5000, NEAR_SWAPS = 50;
    vector<int> nearly(NEAR_N);
    for (int k = 0; k < NEAR_N; k++) nearly[k] = 20 * (k + 1);
    for (int t = 0; t < NEAR_SWAPS; t++) {
        int p = rng() % (NEAR_N - 1);
        swap(nearly[p], nearly[p + 1]);
    }

    cout << endl << "=== Part 4: almost sorted input (n = " << NEAR_N << ", "
         << NEAR_SWAPS << " random neighbour swaps, " << REPEATS << " runs each) ===" << endl;
    cout << left << setw(16) << "Algorithm" << right << setw(12) << "time (ms)"
         << setw(14) << "comparisons" << endl;
    for (int a = 0; a < NUM_ALGORITHMS; a++) {
        long long c = 0;
        double total = 0;
        for (int r = 0; r < REPEATS; r++) total += timedSort(ALGORITHMS[a], nearly, work, c);
```

```
281         cout << left << setw(16) << ALGORITHMS[a].name << right << setw(12) << total / REPEATS
282              << setw(14) << c << endl;
283     }
284
285     return 0;
286 }
```

Output.

```
suh.my@FACLCC6HG6DCQ InClassWork % g++ -std=c++17 -O2 -Wall -o bin/problem07 cpp/Assignment01_Problem07_Suh.cpp && ./bin/problem07
=== Part 1: comparisons on the eight-element inputs ===
Algorithm             A  Sorted  Reversed
Bubble Sort          25       7        28
Insertion Sort       20       7        28
Selection Sort       28      28        28
Quick Sort           14      28        28
Merge Sort           17      12        12
Heap Sort            26      27        24

=== Part 2a: average running time in ms on random arrays (5 runs each) ===
Algorithm          n = 1000    n = 5000   n = 10000  10000/1000  10000/5000
Bubble Sort           0.276       5.831      22.107        80.1        3.79
Insertion Sort        0.077       1.472       5.685        73.8        3.86
Selection Sort        0.789      19.229      73.981        93.7        3.85
Quick Sort            0.015       0.124       0.442        29.0        3.56
Merge Sort            0.059       0.232       0.750        12.6        3.23
Heap Sort             0.023       0.147       0.591        26.0        4.03

=== Part 2b: comparisons on the same random arrays ===
Algorithm          n = 1000    n = 5000   n = 10000  10000/1000  10000/5000
Bubble Sort          498720    12489115    49980465       100.2        4.00
Insertion Sort       252765     6345041    24844387        98.3        3.92
Selection Sort       499500    12497500    49995000       100.1        4.00
Quick Sort            10570       71293      148018        14.0        2.08
Merge Sort             8721       55262      120523        13.8        2.18
Heap Sort             16866      107611      235530        14.0        2.19
Theory, 1000 -> 10000: n^2 grows 100x, n log n about 13.3x
Theory, 5000 -> 10000: n^2 grows 4x,   n log n about 2.16x

=== Part 3: first 10 elements of each sorted array ===
n = 1000
  Bubble Sort     19 176 194 212 250 330 584 668 704 745
  Insertion Sort  19 176 194 212 250 330 584 668 704 745
  Selection Sort  19 176 194 212 250 330 584 668 704 745
  Quick Sort      19 176 194 212 250 330 584 668 704 745
  Merge Sort      19 176 194 212 250 330 584 668 704 745
  Heap Sort       19 176 194 212 250 330 584 668 704 745
n = 5000
  Bubble Sort     12 76 88 96 121 130 138 147 193 209
  Insertion Sort  12 76 88 96 121 130 138 147 193 209
  Selection Sort  12 76 88 96 121 130 138 147 193 209
  Quick Sort      12 76 88 96 121 130 138 147 193 209
  Merge Sort      12 76 88 96 121 130 138 147 193 209
  Heap Sort       12 76 88 96 121 130 138 147 193 209
n = 10000
  Bubble Sort     24 29 39 51 52 64 65 76 85 85
  Insertion Sort  24 29 39 51 52 64 65 76 85 85
  Selection Sort  24 29 39 51 52 64 65 76 85 85
  Quick Sort      24 29 39 51 52 64 65 76 85 85
  Merge Sort      24 29 39 51 52 64 65 76 85 85
  Heap Sort       24 29 39 51 52 64 65 76 85 85
Full check of all 18 sorted arrays: every element is in order

=== Part 4: almost sorted input (n = 5000, 50 random neighbour swaps, 5 runs each) ===
Algorithm          time (ms)   comparisons
Bubble Sort            0.007         14994
Insertion Sort         0.011          5049
Selection Sort        18.740      12497500
Quick Sort             3.241      12367288
Merge Sort             0.169         32025
Heap Sort              0.149        112100
suh.my@FACLCC6HG6DCQ InClassWork %
```

Item 4.

- Bubble, Insertion and Selection Sort grow like n^2. Quick, Merge and Heap Sort grow like $n \log n$.
- Mostly. From 1,000 to 10,000 values the n^2 sorts took 74 to 94 times longer (theory: 100) and the $n \log n$ sorts 13 to 29 times longer (theory: 13.3). The $n \log n$ times are under 1 ms, so they are noisier, but the comparison counts grow almost exactly as predicted: about 100 times for the n^2 sorts and 14 times for the $n \log n$ sorts.

- Large random array: Quick Sort, the fastest here (0.442 ms for 10,000 values), with a random or median-of-three pivot so that sorted input cannot cause its $O(n^2)$ case.
- Almost sorted array: Insertion Sort, which needs about n comparisons when few elements are out of place. On the almost sorted array it made 5,049 comparisons, while Quick Sort with the last-element pivot made 12,367,288.